Name: . . . . . Number: . . . . . .

CSC211 Data Structures and Algorithms

Scatter functions for hashing

Term 1 Tutorial 4 — 4/5 March 2026

Submit today before leaving tutorial

Feel free to consult Gemini, DeepSeek, ChatGPT, Claude, or Grok.

We describe hashing in terms of a table **d[]** that can be indexed with numbers in the range **[1..m]**. For this tutorial we use keys that are strings in Python or elements of the **String** class in Java. The hash table has slots with **m** numbers in the range **[1..m]**. The tut intends to inform you about some ways that scatter functions can be produced, and what properties they have. An *open* hash table does not permit **n** to exceed **m**, because all its data is stored in a one-to-one relationship with **d**. Given the load factor $\boldsymbol{\alpha} = \mathbf{n/m}$, for an open hash table $\boldsymbol{\alpha} \leq 1$. The average search length in an open hash table is $\frac{1}{1-\boldsymbol{\alpha}}$. However, since in a *chained* hash table there may be may more keys **n** than slots **m** in the table, the loading factor may exceed 1. Now the loading factor indicates the average length of the chains. The average search length for a successful search in a chained hash table is half of this. When the first element is not stored in the head node, this averagge search length becomes $\frac{\boldsymbol{\alpha}}{2} + 1$. When the search is unsuccessful the entire chain at the index is traversed, giving $\boldsymbol{\alpha}$.

K

1. The polinomial $P_n(x)$ can be written as $P_n(x) = a_0 + a_1x + a_2x^2 + \ldots + a_{n-1}x^{n-1} + a_nx^n$ or rather as $P_n(x) = a_nx^n + a_{n-1}x^{n-1} + \ldots + a_2x^2 + a_1x + a_0$ making its relationship with numbers clearer. When the values $\{a_k\}_{k=0}^{n}$ are limited to (a) the set $\{0, 1\}$, (b) the set $\{0, 1, \ldots, 7\}$, or (c) the set $\{0, 1, \ldots, 9\}$, these polinomials $P_n(2)$ represent *binary numbers*, (b) $P_n(8)$, *octal numbers*, or (c) $P_n(10)$, *decimal numbers*. An efficient way to evaluate polinomials is by using Horners's rule. Horner's rule starts by initializing $P \leftarrow a_n$, the next step is $P \leftarrow P \times x + a_{n-1}$, ..., the third last step is $P \leftarrow P \times x + a_2$, the second last step is $P \leftarrow P \times x + a_1$ and the last step is $P \leftarrow P \times x + a_0$. Get a function called Horner's rule working on a computer in Python. Use **def P(x, A):** to define the function, where **x** can be any value such as 2, 8, 10, 37 or 97, and **A** is an appropriate string of digits when $x \leq 10$ or otherwise characters such as letters. Note that **P(x, A)** always returns an integer. You may find this code snippet useful.

```
def toArrayOfInts(parA):
        A = [ord(a)-48 for a in parA]
        A.reverse()
        return A
```

Use your program code to calculate the following; double check your numbers.

(a) **P(2, "100000000000")** . . . 1024

(b) **P(8, "77777")** 4095

(c) **P(10, "1234567890")** 1234567890

(d) **P(37, "Something else")** . . . . . 2414771665....

Horners

```
P(x, A)
p=0
for digit in A:
    if digit.isdigit():
```

Find code w/ explanation in Vault

Rule

```
        a = int(digit)
    else:
        a = ord(digit) - ord('a') + 10
    P = (P * x + a)
return P
```

2. Now alter **P(x, A)** to **h(x, A, m)** where the Horner step $P \leftarrow P \times x + a_k$ is replaced by $P \leftarrow (P \times x + a_k)\%m + 1$. in order to create a hash index that lies in the range **[1..m]**. Use your program code to calculate the following; double check your numbers.
   (a) **h(2, "100000000000", 137)** 65
   (b) **h(8, "77777", 137)** 122
   (c) **h(10, "1234567890", 137)** 161
   (d) **h(37, "Something else", 137)** 61

3. Give a listing of your Python code for **h(x, A, m)**

def h(x, A, m):

```
h(x, A, m)
P = 0
for digit in A:
    if digit.isdigit():
        a = int(digit)
    else:
        a = ord(digit) - ord('a') + 10
    P = (P * x + a) % m + 1
return P
```